“시간이 아깝지 않은”

SK네트웍스 Family AI캠프 교재3

강사 이영배

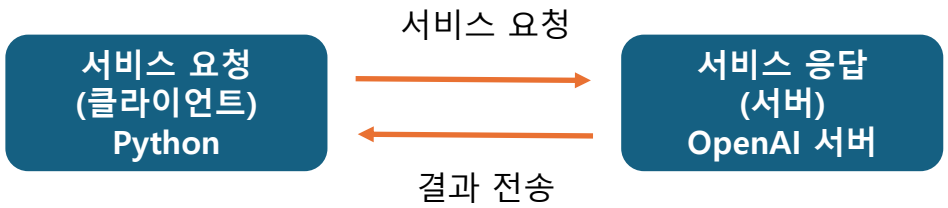
05. 데이터 수집

데이터 수집 방법

- Open API
- Web 크롤링(crawling)

오픈 API를 이용한 데이터 수집

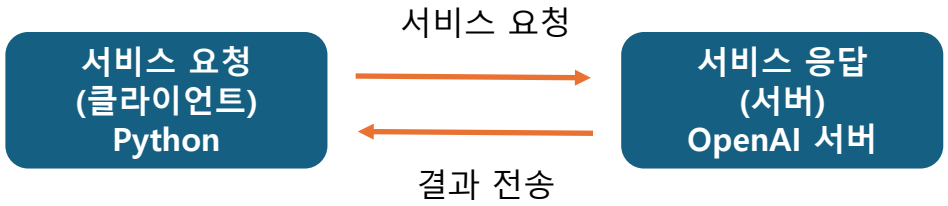
- API(application programming interface) 개념
  1) 서로 다른 컴퓨터 프로그램끼리 대화할 수 있게 돕는 메신저이자 통신 규칙
  2) 반드시 OpenAI 등 서비스 업체에서 제공하는 규칙대로 구현하고 실행(API 매뉴얼)
  3) 종류: REST API(웹 API), 라이브러리 API



오픈 API를 이용한 데이터 수집

- JSON (JavaScript Object Notation) 정리

1) 개념: 서버와 클라이언트가 서로 데이터를 주고받을 때 사용하는 , 키-값 쌍으로 이루어진 텍스트 기반의 데이터 표준 형식




```
{
  "books": [
    {
      "book_id": 1,
      "title": "시간의 경계",
      "author": "이수진",
      "genre": "SF"
    },
    {
      "book_id": 2,
      "title": "사라진 기억",
      "author": "김민재",
      "genre": "미스터리"
    },
    {
      "book_id": 3,
      "title": "바람의 노래",
      "author": "정하늘",
      "genre": "판타지"
    }
  ]
}
```


2) python dict 자료형과 비교

	Python dict 자료형	JSON 문자열
구조	{key : value}	{key : value}
자료형	dict 자료형	string 자료형
따옴표 구분	작은따옴표('), 큰따옴표(") 모두 허용	반드시 큰따옴표(")만 사용
URL을 통한 데이터 전송 가능성	**불가능**	**가능**

오픈 API를 이용한 데이터 수집

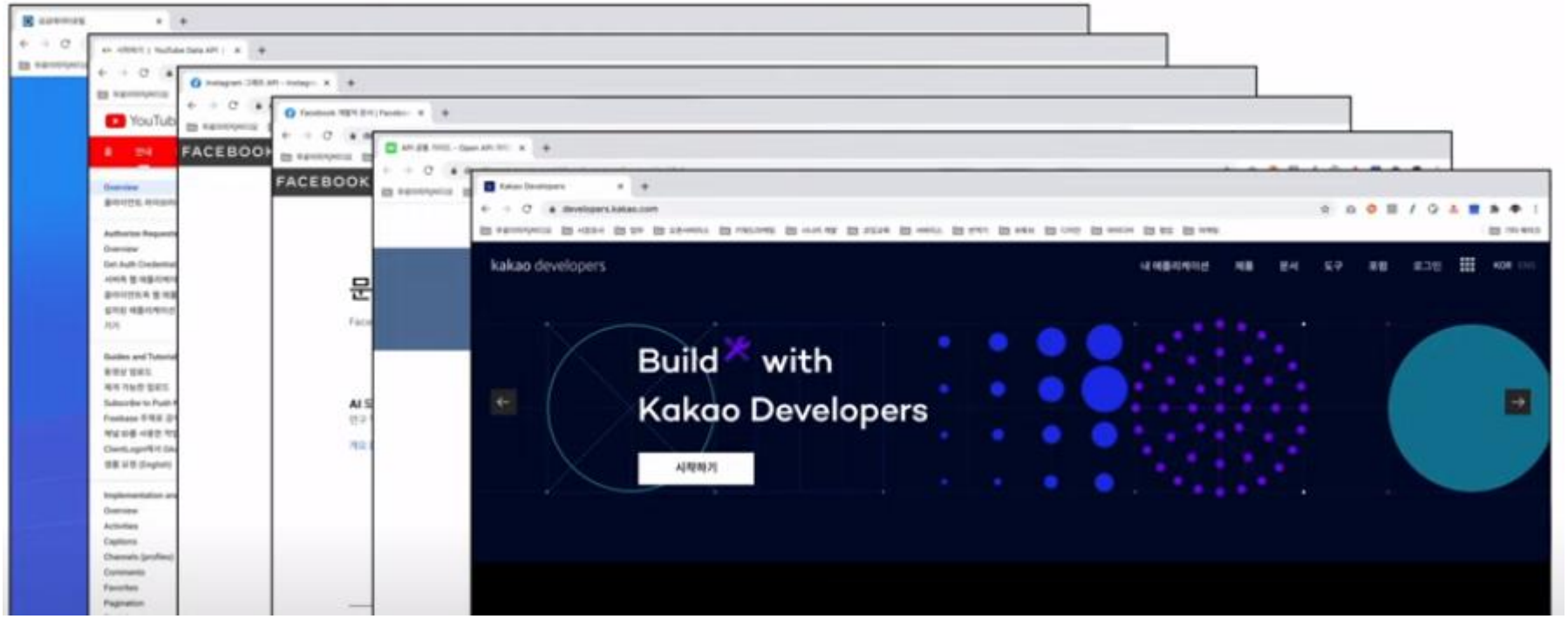
3) JSON 문자열을 python dict 자료형으로 변환하기

▶ json.loads() 함수 사용

```
import json
python_dict = json.loads(json_string)
```

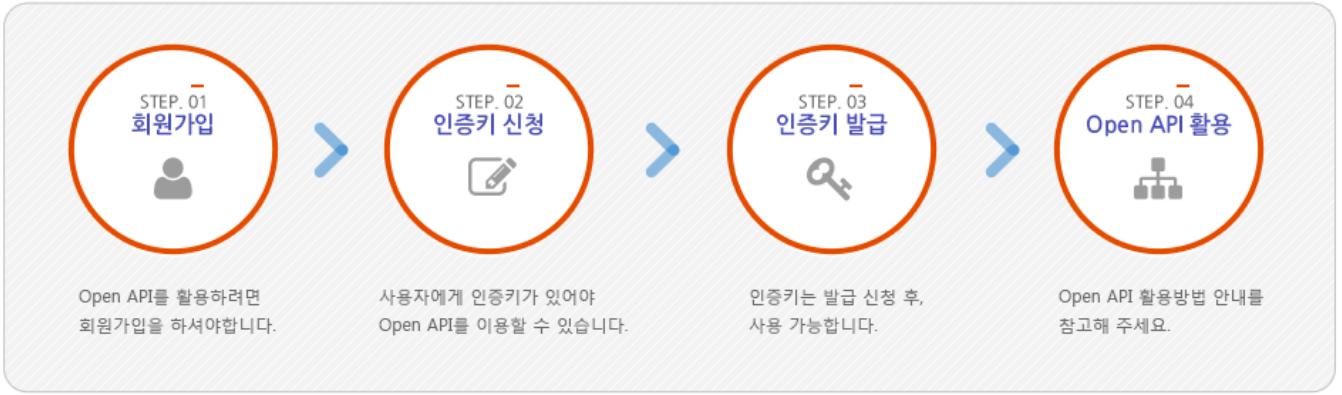
오픈 API를 이용한 데이터 수집

- API(application programming interface) 종류
  1) Private API(비공개 API): 조직 내에서만 사용 가능한 API
  2) 오픈 API: 누구나 사용 가능한 API



오픈 API를 이용한 데이터 수집

- 개요: 도서관 정보나루에서 제공하는 API 활용, 20대가 가장 좋아하는 도서 수집
- 수집 과정
  1) 도서관 정보나루 접속
  2) 메인 페이지 메뉴에서 [데이터 활용] 탭 클릭
     ▶ [Open API 활용방법] 탭 클릭
     - 정보나루에서 제공하는 API에 관한 내용이 정리되어 있음
     - 이용안내 항목 옆의 [API매뉴얼다운로드] 버튼 클릭 : 매뉴얼 다운로드
     - 하단에 이용절차 제시



오픈 API를 이용한 데이터 수집 실습

3) 회원 가입

4) API 인증키 발급 → 사이트 오른쪽 상단에 있는 [마이페이지] 클릭

▶ 인증키 정보 항목

- [개인정보 수집 및 이용에 관한 안내]의 [선택항목 : 동의] 체크
- [사용목적] 에서 '목적' 선택
- [인증키] – [인증키발급] 버튼 클릭 → 인증키 발급 신청 완료
- [인증상태]: API키 발급 완료 시 '승인완료(사용중)' 표시

5) 다운로드 한 매뉴얼 → 연령별 대출 데이터 제공 API 확인

① API 매뉴얼 7page → 3.인기대출도서 조회 → 연령별 데이터 조회 가능 문구 확인

② API 매뉴얼 7page → 활용 예제 참조 → API 호출 URL 작성

오픈 API를 이용한 데이터 수집 실습

6) 호출 URL 작성

호출 URL 형식
http://data4library.kr/api/loanItemSrch?authKey=[발급받은키]&startDt=2022-01-01&endDt=2022-03-31&gender=1&age=20&region=11;31&addCode=0&kdc=6&pageNo=1&pageSize=10

① 기본 URL : http://data4library.kr/api/loanItemSrch

② Query String: 파리미터 정리

- authKey: 인증키
- startDt: 검색 시작 일자, 2025년 01월 01일로 지정
- endDt: 검색 종료 일자, 2026년 06월 30일로 지정
- age: 연령대, 20으로 지정, '20대' 대출 도서를 조회
- format: format=JSON으로 지정, 지정하지 않으면 XML 문서로 데이터 전달

③ 호출 URL 작성 방법

- 파라미터와 값은 =로 연결, 파라미터와 파라미터는 &로 연결
- 기본 URL과 Query String은 ?로 연결

오픈 API를 이용한 데이터 수집 실습

7) 호출 URL을 통한 데이터 수집

① 호출 URL 완성 :

http://data4library.kr/api/loanItemSrch?authKey=[인증키]&startDt=2025-01-01&

endDt=2025-06-30&format=json

② 크롬 브라우저를 이용한 호출 결과



오픈 API를 이용한 데이터 수집 실습

- python으로 API 호출하기

1) requests 라이브러리 사용

: python에서 URL(http://~)을 통해서 데이터를 받을 수 있는 라이브러리

2) requests 라이브러리 사용법

① 설치: pip install requests

② 라이브러리: import requests

③ 호출 URL 완성

```
url = "" http://data4library.kr/api/loanItemSrch?authKey=[인증키]&startDt=2025-01-01&
        endDt=2025-06-30&format=json """
```

④ requests.get() 함수 사용 → 데이터 요청 & JSON 문자열 형태의 데이터 수령

```
data = requests.get(url).text
```

오픈 API를 이용한 데이터 수집 실습

- 도서 정보 추출하기

1) JSON 문자열을 python dict 자료형으로 변환하기

: json.loads() 함수 사용

```
import json
python_dict = json.loads(json_string)
```

2) python dict 전용 함수 사용 → 도서 정보 추출

3) 추출한 도서 정보 → DataFrame 변환 → csv 파일로 저장

오픈 API를 이용한 데이터 수집 실습

생성형 AI와 Open API를 이용한 데이터 수집 실습

- 수집 목표: 2025년 1월 1일부터 2026년 6월 30일까지 20대가 지역에 상관 없이 가장 많이 대출해간 인기 도서 1위부터 200위까지 200권의 정보 수집 및 CSV 파일로 저장하기

- 수집 과정
  1) 생성형 AI에 전달할 정보 수집: 예) API 매뉴얼
  2) 생성형 AI를 이용하여 수집 및 CSV 파일 생성 프롬프트 작성
  3) 생성된 코드 실행 및 검증

생성형 AI와 Open API를 이용한 데이터 수집 실습

Open API 수집 데이터의 MySQL DB 저장 및 조회 실습

- 실습 주제

  1) 도서관 정보나루 Open API로 수집한 '2025~2026년 20대 인기 도서 200위' CSV 데이터를

  2) MySQL testdb.book_table에 저장하고, SQL문으로 데이터를 검증·조회한다.

- 학습 목표

  1) CSV 파일의 데이터 구조와 컬럼 타입을 분석하여 최적의 MySQL 테이블(DDL)을 설계할 수 있다.

  2) MySQL Workbench의 Table Data Import Wizard를 활용해 CSV 데이터를 안정적으로 밀어 넣을 수 있다.

  3) SELECT, WHERE, ORDER BY, GROUP BY 명령어로 저장된 인기 도서 데이터를 조회할 수 있다.

MySQL DB 생성 및 book_table 설계 (DDL)

- testdb 데이터베이스 준비

```
CREATE DATABASE IF NOT EXISTS testdb
# 데이터베이스 생성 시 이모지/다국어 지원 설정 (최초 1회만)
DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

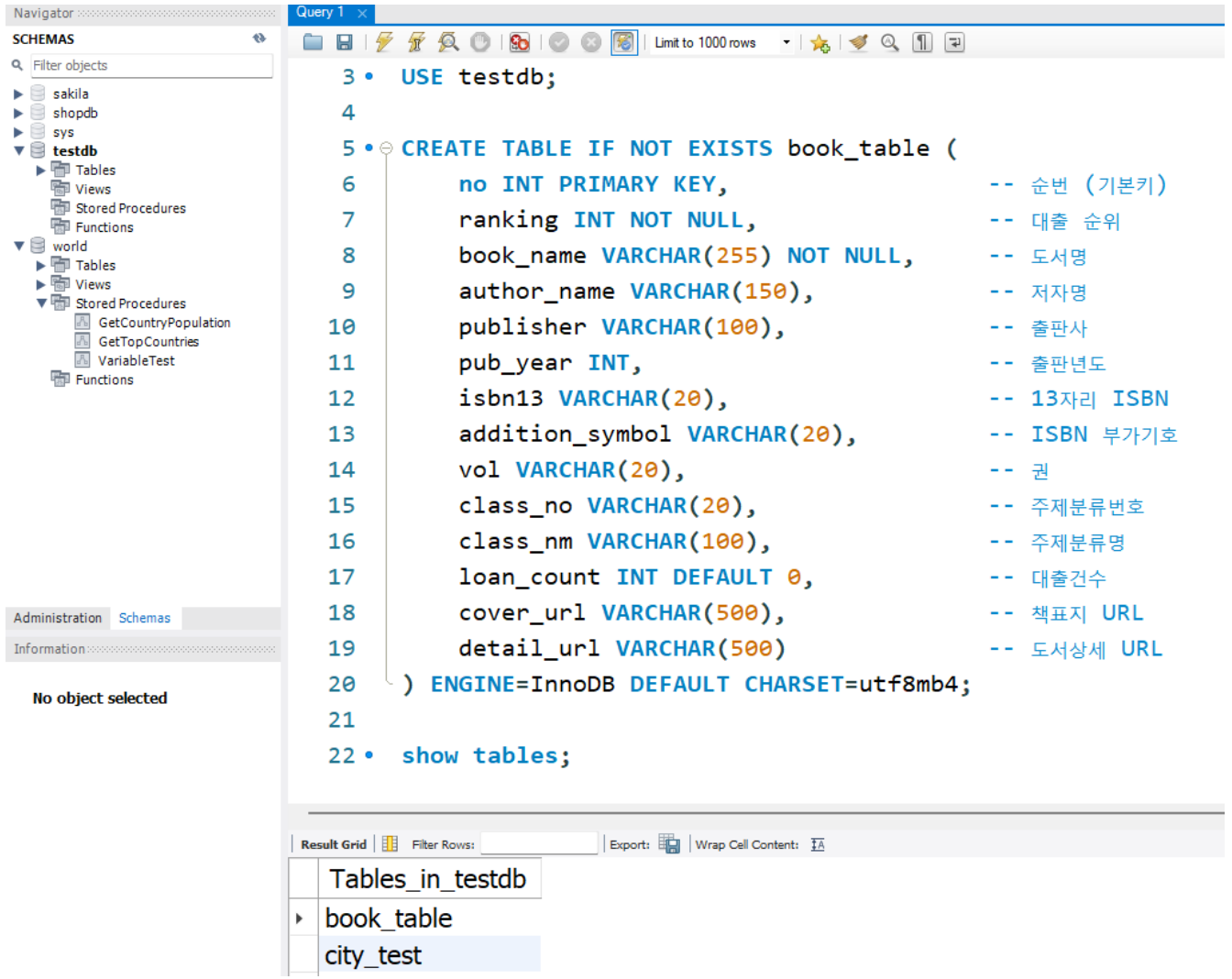
MySQL DB 생성 및 book_table 설계 (DDL)

- book_table 생성 구문;

```
USE testdb;

CREATE TABLE IF NOT EXISTS book_table (
    no INT PRIMARY KEY,                 -- 순번 (기본키)
    ranking INT NOT NULL,               -- 대출 순위
    book_name VARCHAR(255) NOT NULL,    -- 도서명
    author_name VARCHAR(150),           -- 저자명
    publisher VARCHAR(100),             -- 출판사
    pub_year INT,                       -- 출판년도
    isbn13 VARCHAR(20),                 -- 13자리 ISBN
    addition_symbol VARCHAR(20),        -- ISBN 부가기호
    vol VARCHAR(20),                    -- 권
    class_no VARCHAR(20),               -- 주제분류번호
    class_nm VARCHAR(100),              -- 주제분류명
    loan_count INT DEFAULT 0,           -- 대출건수
    cover_url VARCHAR(500),             -- 책표지 URL
    detail_url VARCHAR(500)             -- 도서상세 URL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

MySQL DB 생성 실습



CSV 파일 데이터 MySQL 테이블에 적재(Import)

- Workbench Table Data Import Wizard 실행 절차

1) SCHEMAS 탭 이동: Workbench 왼쪽 Navigator에서 `testdb` 확장

2) 마법사 실행: `Tables` 우클릭 → `Table Data Import Wizard` 선택

3) 파일 경로 지정: `Browse` 버튼 클릭 → `2025_2026_20대_인기도서_200위.csv` 선택 후 [Next]

4) 테이블 지정:

: `Use existing table` 선택 → `book_table` 지정 → [Next]

5) 컬럼 매핑 (Field Mappings)

: CSV의 한글 헤더(`순번`, `도서명` 등)와 DB 컬럼(`no`, `book_name` 등) 1:1 매핑 상태 확인

6) 실행: [Next] → [Next] 눌러 200건 데이터 적재 완료 후 [Finish];

데이터 수집 및 수량 검증 실습

- 검증 포인트

  : Open API로 가져온 200건의 인기 도서 데이터가 빠짐없이 DB에 들어왔는지 확인

- 실습 스크립트

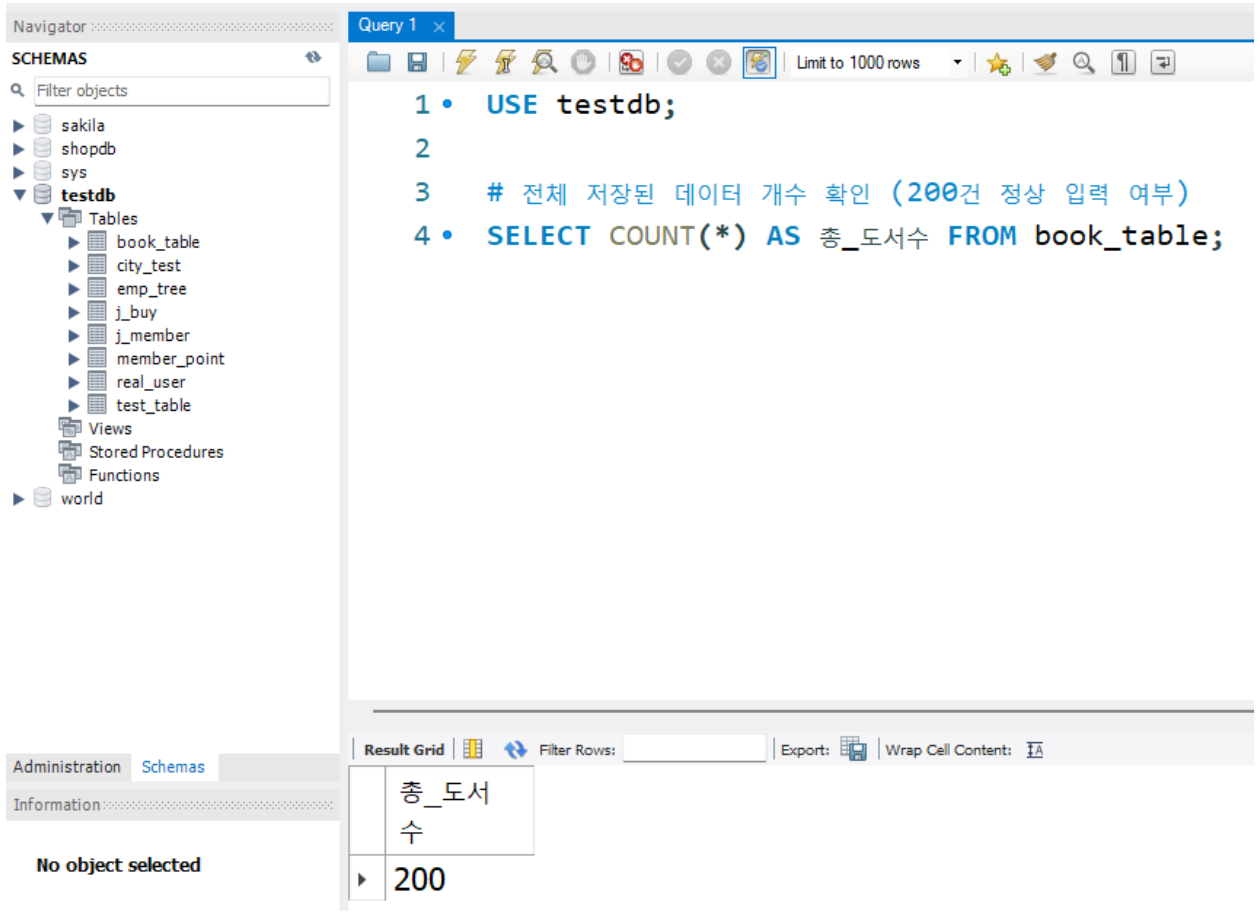
```
USE testdb;

# 전체 저장된 데이터 개수 확인 (200건 정상 입력 여부)
SELECT COUNT(*) AS 총_도서수 FROM book_table;

# 상위 10위 인기 도서 기본 정보 확인
SELECT ranking AS 순위, book_name AS 도서명, author_name AS 저자, publisher AS 출판사, loan_count AS 대출건수
FROM book_table
ORDER BY ranking ASC
LIMIT 10;
```

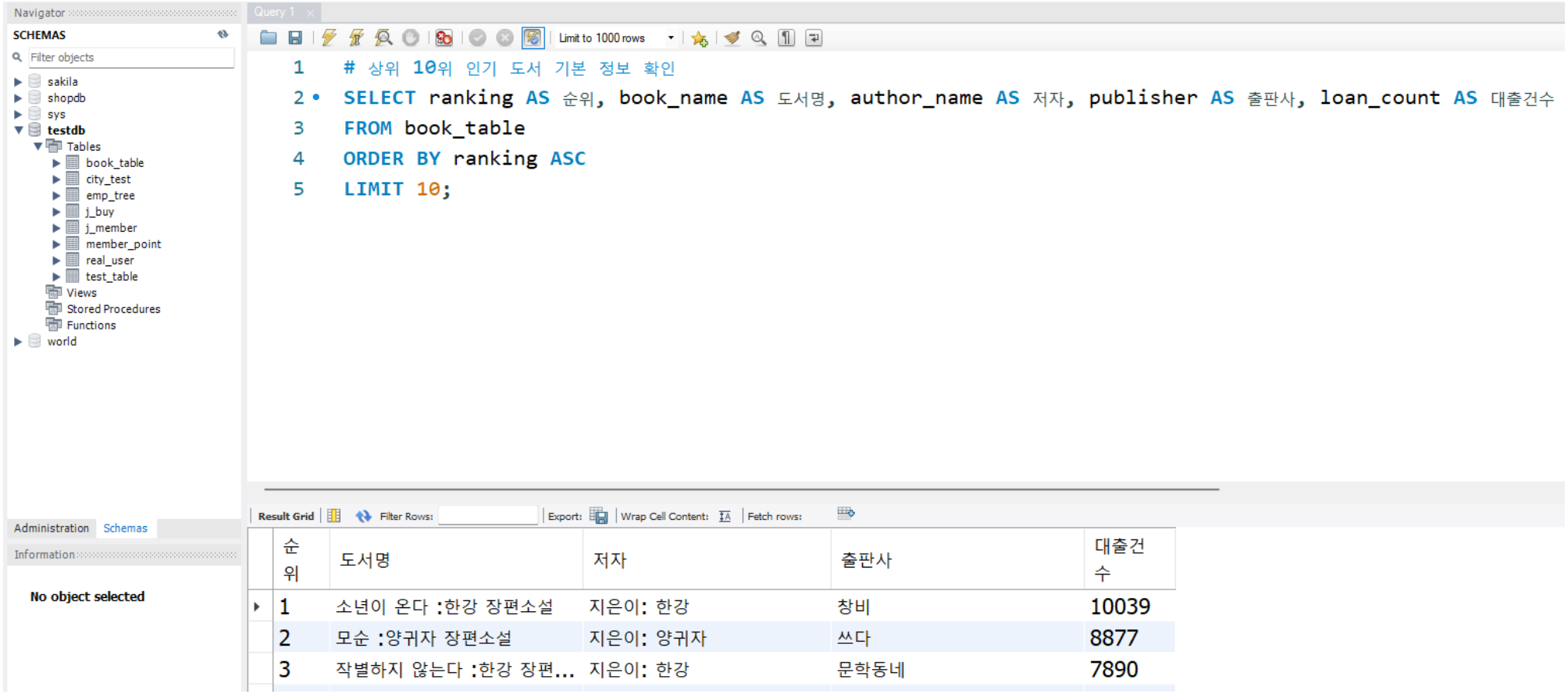
데이터 수집 및 수량 검증 실습

- 전체 저장된 데이터 개수 확인 (200건 정상 입력 여부)



데이터 수집 및 수량 검증 실습

- 상위 10위 인기 도서 기본 정보 확인



조건별 도서 검색 및 필터링 실습

- 검증 포인트

  : 특정 작가(예: 한강)의 도서 검색, 대출건수 상위 도서 추출

- 실습 스크립트

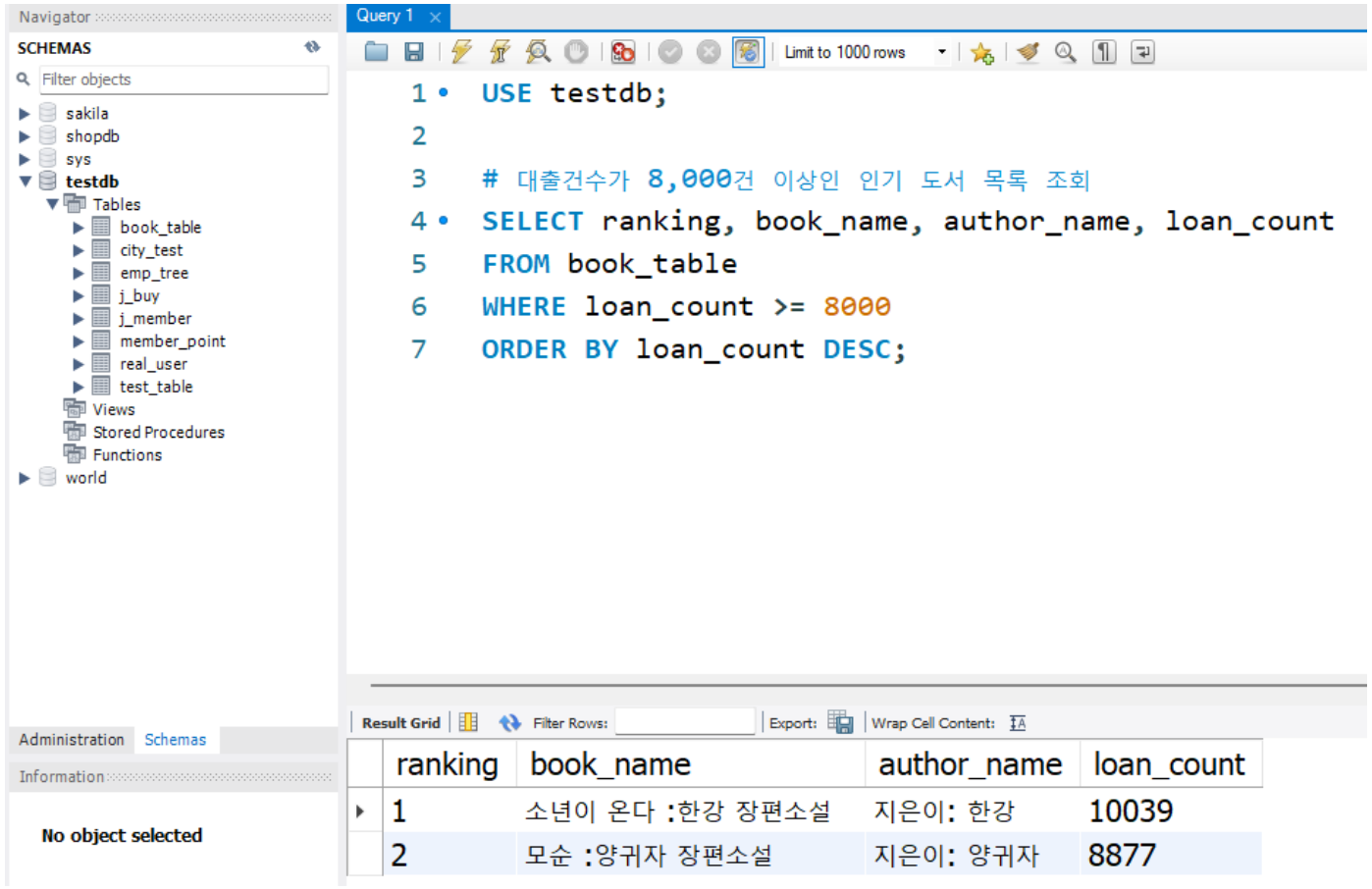
```
USE testdb;

# 대출건수가 8,000건 이상인 인기 도서 목록 조회
SELECT ranking, book_name, author_name, loan_count
FROM book_table
WHERE loan_count >= 8000
ORDER BY loan_count DESC;

# 저자명에 '한강'이 포함된 노벨문학상 수상 작가 도서 검색
SELECT ranking, book_name, publisher, pub_year, loan_count
FROM book_table
WHERE author_name LIKE '%한강%'
ORDER BY ranking ASC;
```

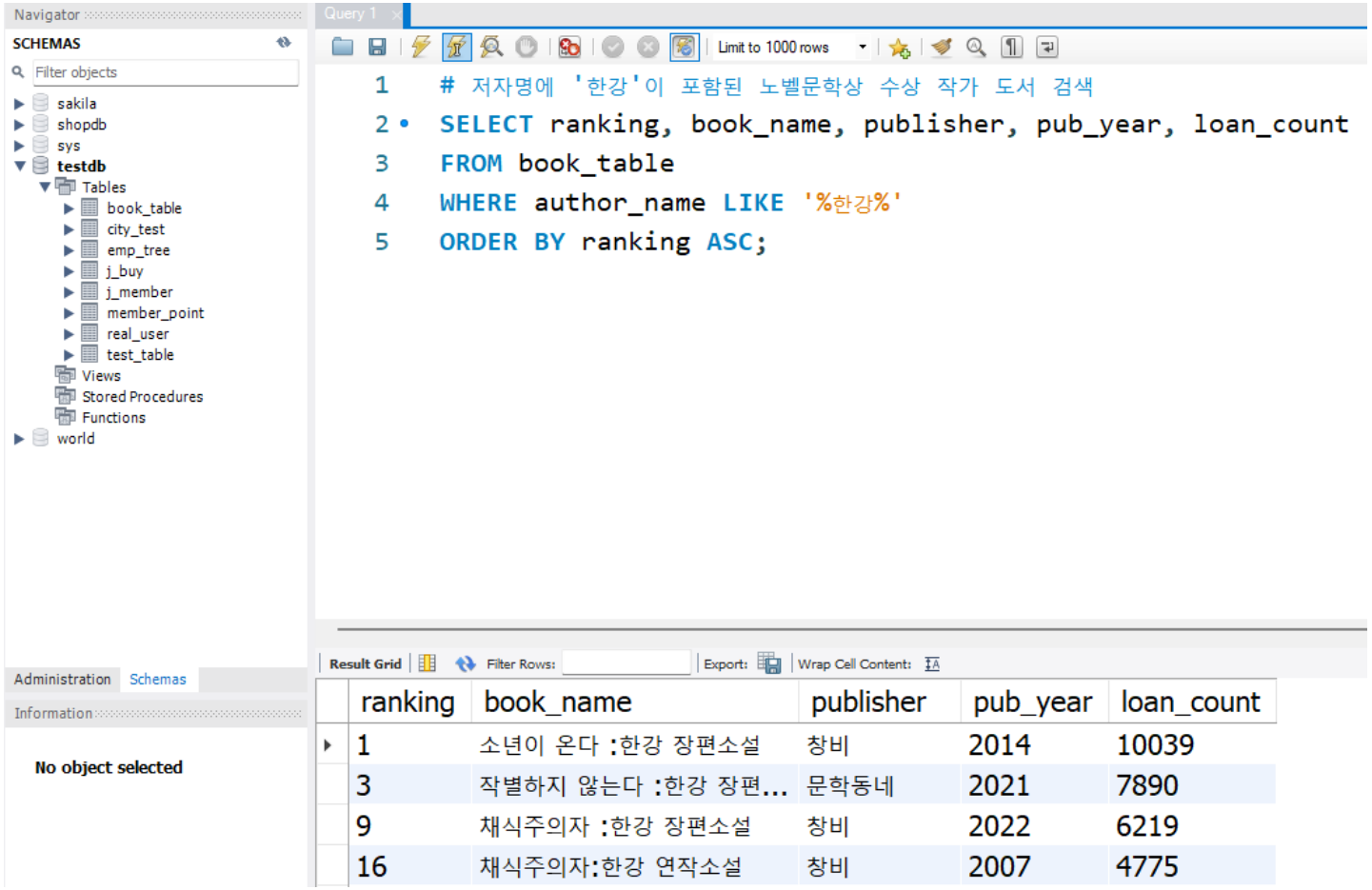
조건별 도서 검색 및 필터링 실습

- 대출건수가 8,000건 이상인 인기 도서 목록 조회



조건별 도서 검색 및 필터링 실습

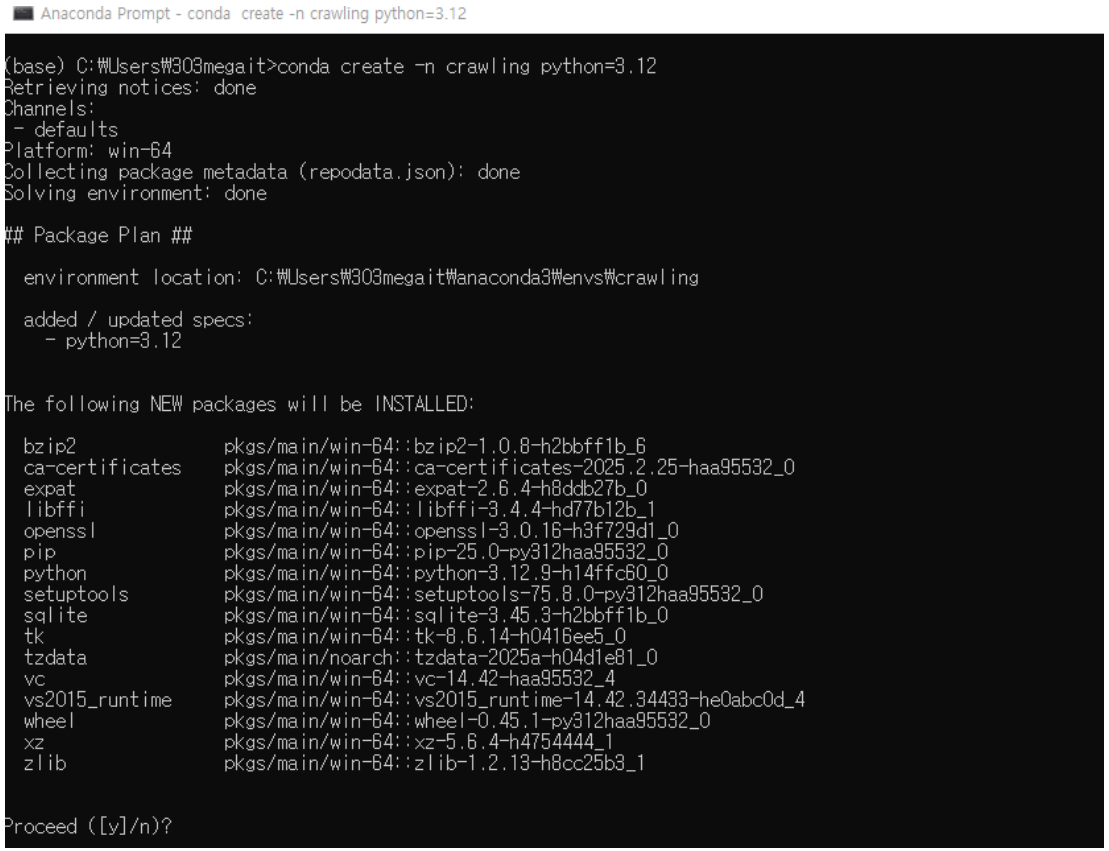
- 저자명에 '한강'이 포함된 노벨문학상 수상 작가 도서 검색



크롤링을 이용한 웹 데이터 수집: 가상 환경 설치

1. 윈도우 시작(또는 찾기) → anaconda 입력 → Anaconda Prompt 실행

2. 가상 환경 설치 : conda create –n crawling python=3.12



크롤링을 이용한 웹 데이터 수집: 가상 환경 설치

3. 가상 환경 활성화 : conda activate crawling

4. 필요한 라이브러리 설치: pip install beautifulsoup4 selenium webdriver-manager pandas

5. miniconda3 설치 폴더 → 새 폴더 생성 : work2

6. Visual Studio Code의 메뉴에서 'File' >> 'Open Folder'를 선택, work2 폴더 선택

7. CTRL+SHIFT+P를 입력, 'Python: Select Interpreter' 선택, crawling 인터프리터 선택

크롤링을 이용한 웹 데이터 수집

- 크롤링(crawling)
  1) 개념: 웹 페이지의 정보를 자동화된 방법으로 수집하고 데이터를 추출하여 저장하는 기술
  2) 종류: 정적 크롤링, 동적 크롤링

비교 항목	정적 크롤링	동적 크롤링
데이터 형태	변하지 않는 고정된 데이터	상호작용으로 변하는 실시간 데이터
수집 대상 데이터 예시	뉴스 기사 본문, 블로그 포스트, 백과사전 텍스트	인스타그램 피드(무한 스크롤), 클릭해야 열리는 댓글
수집 속도	매우 빠름	상대적으로 느림
브라우저 구동	구동하지 않음 (HTML만 요청)	브라우저(크롬 등) 실제 구동
수집 가능 범위	정적 데이터만 수집 가능	동적/정적 데이터 모두 수집 가능
사용 라이브러리	Requests, BeautifulSoup	Selenium

크롤링을 이용한 웹 데이터 수집

- HTML
  - 개념

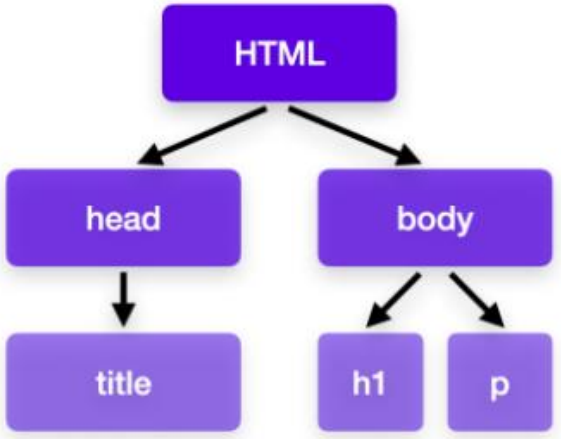
1) Hypertext Markup Language의 약자로 웹 페이지를 만드는 데 사용하는 언어
2) 브라우저를 통해 보는 페이지는 모두 HTML로 작성
3) 웹 페이지: 사용자가 보는 화면, 결과물 ↔ 웹 페이지를 만들기위한 설계도(HTML 문서)
4) 태그(tag)
   ① HTML 문서 구성 기본 단위, 웹페이지의 구조와 콘텐츠를 정의하는 기본적인 단위
   ② 여러 가지 태그를 이용하여 제목이나 문단, 표, 이미지 등을 구분하고 표현
   ③ 구조
      • 시작 태그(<>)와 종료 태그(</>)가 한 쌍을 이루어 콘텐츠(텍스트, 이미지 등)를 감싸고 있음
      • <태그이름>콘텐츠</태그이름>

크롤링을 이용한 웹 데이터 수집

- HTML
  - 웹 페이지의 기본 구조

1) <!DOCTYPE html> 해당 HTML문서가 HTML5 표준 형식임을 선언
2) <html> 태그: HTML 문서의 시작과 끝을 알려주는 태그, 해당 태그 안에서 HTML 문서를 크게 <head> 태그와 <body>태그로 영역을 나눔
4) <head> 태그: 사용자 눈에는 보이지 않지만, 브라우저가 해당 웹 페이지를 해석하는데 필요한 여러가지 부가 정보(metadata)가 표시되는 곳
5) <body> 태그: 사용자 눈에 보이는, 즉 브라우저를 통해 표시되는 모든 콘텐츠(텍스트 등)가 위치하는 곳

```
<!DOCTYPE html>
<html>
<head>
  <title>페이지 타이틀</title>
</head>
<body>
  <h1>제목</h1>
  <p>들어갈 문장</p>
</body>
</html>
```



크롤링을 이용한 웹 데이터 수집

- HTML 태그의 종류
  - a 태그

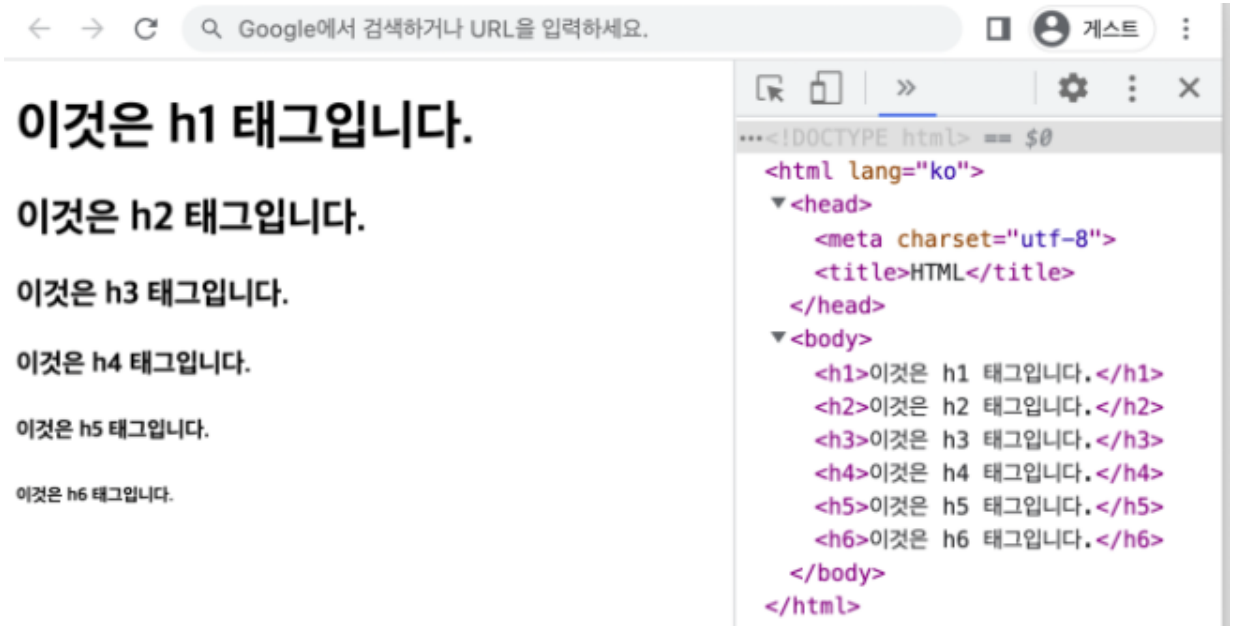
1) 다른 웹페이지나 같은 페이지 내 특정 위치 등으로 연결되는 하이퍼링크(hyperlink)를 정의

2) 연결될 링크의 주소를 href 속성을 통해 지정, 링크에_표시될_내용

3) 예시) 네이버로 이동하기

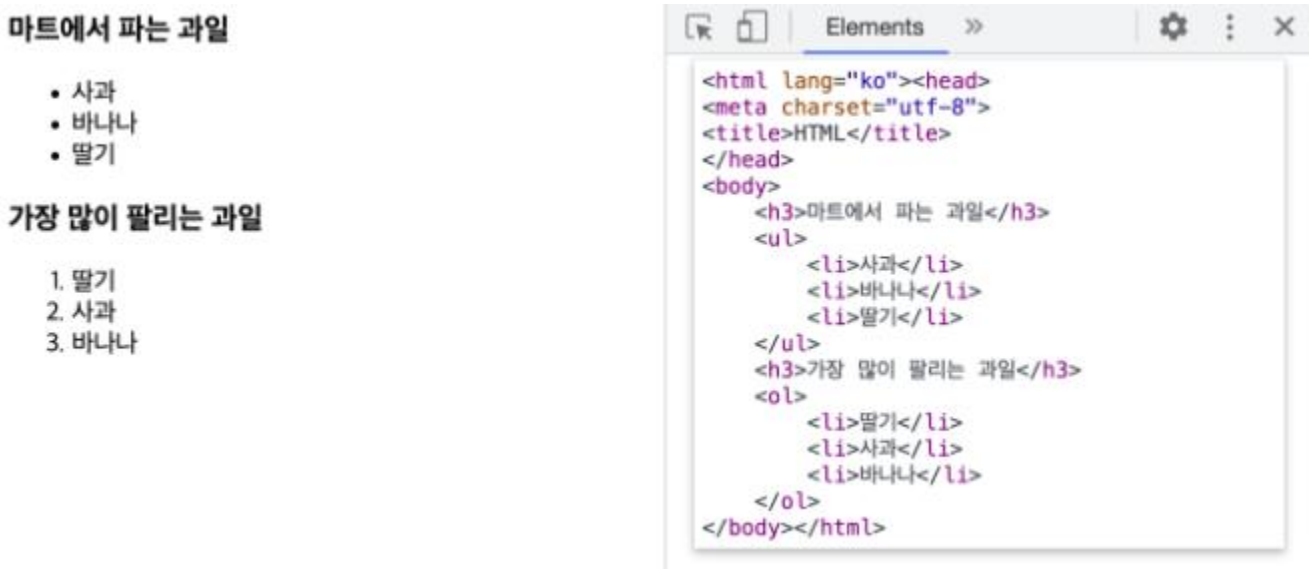
크롤링을 이용한 웹 데이터 수집

- HTML 태그의 종류
  - h 태그
    1) <h1> ~ <h6> 태그가 존재
    2) 다양한 크기로 문서의 제목(Heading) 정의



크롤링을 이용한 웹 데이터 수집

- HTML 태그
  - ul, ol, li 태그
    1) 태그: unordered list의 약자, 순서가 없는 불릿 형태로 작성된 목록 정의
    2) 태그: ordered list의 약자, 숫자를 이용하여 순서가 있는 목록 정의
    3) 태그: <ul> 또는 <ol> 태그의 자식 요소로 사용, 각 목록의 개별 항목들 정의



크롤링을 이용한 웹 데이터 수집

- HTML 태그
  - table 태그: 웹 페이지에서 표 형태의 데이터(tabular data)를 정의할 때 사용

1) <table> 태그: 표의 시작과 끝을 정의하는 부모 태그
2) <tr> 태그: 표의 한 행을 정의, 자손으로 th나 td가 반드시 있어야 한다.
3) <th> 태그: 표의 행 또는 열의 제목 정의, 텍스트가 굵고 가운데 정렬되는 특징
4) <td> 태그: 표의 실제 데이터(행의 내용)를 정의

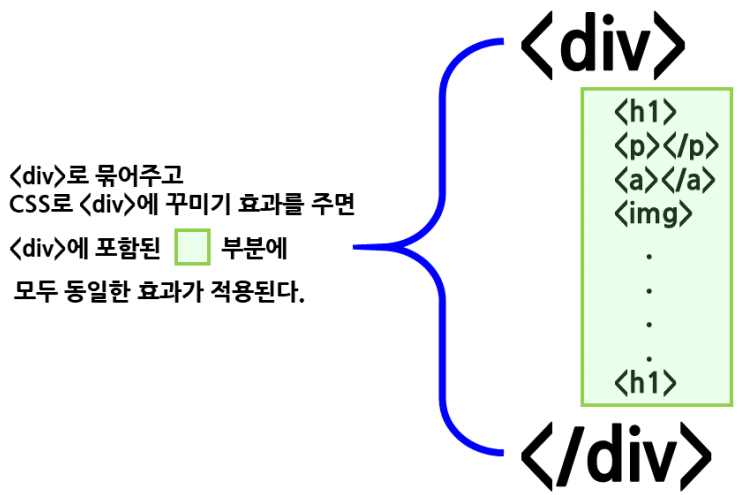
```
<table>
<tr>
<th>날짜</th>
<th>이름</th>
<th>성별</th>
</tr>
<tr>
<td>11/15</td>
<td>김우직</td>
<td>남</td>
</tr>
<tr>
<td>11/14</td>
<td>박수진</td>
<td>여</td>
</tr>
</table>
```

날짜	이름	성별
11/15	김우직	남
11/14	박수진	여

크롤링을 이용한 웹 데이터 수집

- HTML 태그
  - div 태그

1) html 문서안에 존재하는 여러 개의 태그들을 라는 큰 단위로 묶어주는 컨테이너(주머니) 역할 수행
2) 큰 단위인 로 묶어주면, 나중에 css로 에만 수식을 해도 태그 안에 속해 있는 태그들이 모두 같은 명령을 받아 동일하게 적용
3) 여러 태그들을 묶어주는 컨테이너 역할이기 때문에 태그 자체로는 시각적인 변화를 나타낼 수 없음



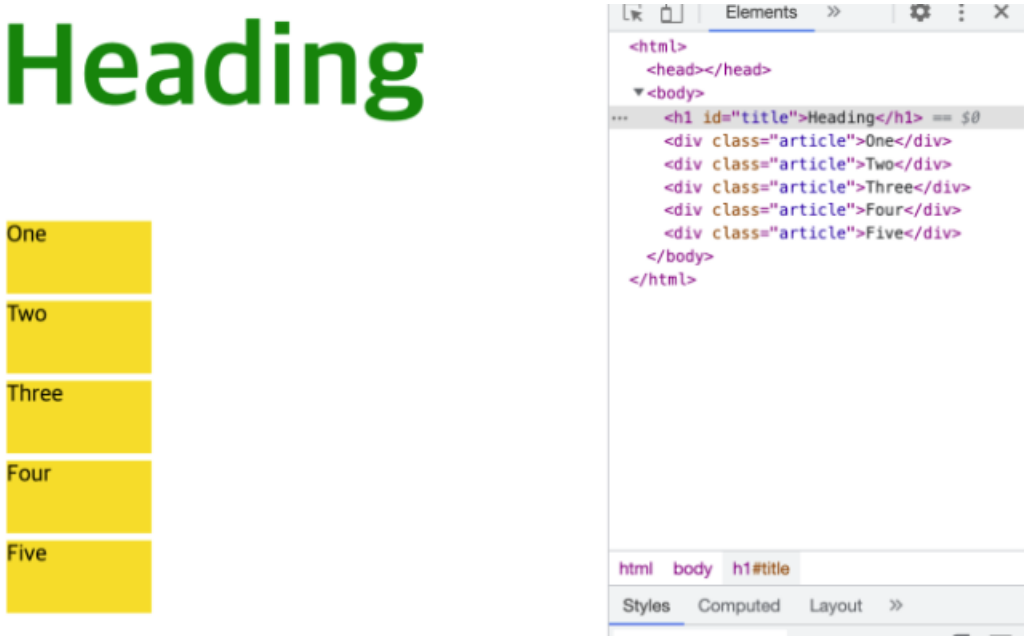
크롤링을 이용한 웹 데이터 수집

- id, class 속성
  - 개념

1) 특정 태그에 시각적 효과를 주고 싶을 때, 해당 태그에 id 속성과 class 속성을 지정해 줄 수 있음

2) id 속성: 서로 다른 태그(또는 같은 태그)에 똑같은 id 중복 사용 불가능, 유일한 요소 지정

3) class 속성: 하나의 class를 여러 태그에 적용 가능(중복 사용 가능)



크롤링을 이용한 웹 데이터 수집

- CSS

1) 개념: HTML 문서를 다양한 스타일로 디자인하는데 사용되는 언어

```
/* red라는 클래스 선택자를 정의합니다. */
    .red {
        color:#ff0000;
    }
/* blue라는 ID 선택자를 정의합니다. */
    #blue {
        color:#0000ff;
    }
/* point라는 클래스 선택자를 정의합니다.
    .point {
        background-color:#000;
        color:#fff;
```

2) HTML 요소를 식별하는 도구(식별자)

① id: 단 하나만 지정하는 '고유 식별자'

② class: 여러 개를 묶을 수 있는 '그룹 식별자

정적 크롤링 : BeautifulSoup 라이브러리를 이용한 웹 데이터 수집

- BeautifulSoup 기본 개념

1) 파이썬에서 HTML과 같은 문서를 파싱(parsing)하고, 원하는 데이터를 쉽게 추출할 수 있는 기능을 제공하는 라이브러리

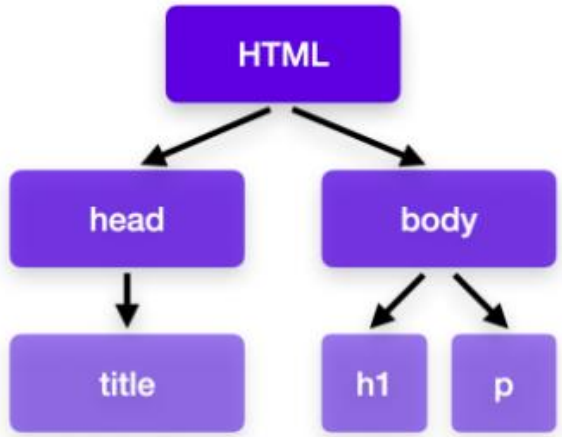
2) 파싱(Parsing) : BeautifulSoup은 웹 페이지를 구성하고 있는 HTML 문서의 코드를 분석하고 구조화하여, 마치 나무처럼 계층적인 형태로 데이터를 표현하는 것

3) 태그(Tag)와 속성(Attribute)

① HTML은 태그와 속성으로 이루어져 있음

② BeautifulSoup은 이러한 태그와 속성을 이용하여 특정 요소에 접근하고 데이터를 추출할 수 있음

```
<!DOCTYPE html>
<html>
<head>
  <title>페이지 타이틀</title>
</head>
<body>
  <h1>제목</h1>
  <p>들어갈 문장</p>
</body>
</html>
```



정적 크롤링 : BeautifulSoup 라이브러리를 이용한 웹 데이터 수집

- BeautifulSoup 기본 사용법
  1) HTML 가져오기
  2) 원하는 태그 찾기
  3) 텍스트 추출하기

```
import requests
from bs4 import BeautifulSoup

# 1단계: 웹페이지 HTML 가져오기
url = "https://example.com"
html = requests.get(url).text

# 2단계: BeautifulSoup 객체 생성 (HTML 분석 시작)
soup = BeautifulSoup(html, 'html.parser')

# 3단계: 원하는 데이터 추출하기
tags = soup.select('CSS선택자')
target = tags[1]
text = target.get_text()
```

정적 크롤링 : BeautifulSoup 라이브러리를 이용한 웹 데이터 수집

- 데이터 추출 핵심 기술

1) select() 함수

① CSS 선택자를 사용하여 복잡한 구조의 데이터도 효율적으로 추출할 수 있음

② 사용법 : soup.select('CSS 선택자')

2) CSS 선택자

① HTML 문서에서 특정 태그를 지정하는 패턴

② 이를 통해 특정 요소나 여러 요소를 정확히 찾아내어 디자인이나 레이아웃을 변경할 수 있음

③ 가장 흔하게 사용되는 기본적인 형태

- 태그(요소) 선택자 (tag): 특정 태그 이름을 가진 모든 요소를 선택합니다 (예: p, h1)
- 클래스 선택자 (.class): 특정 class 속성 값을 가진 요소를 모두 선택합니다 (예: .box)
- 아이디 선택자 (#id): 문서 내에서 유일한 id 속성 값을 가진 요소를 선택합니다 (예: #header)

3) 태그 내부의 텍스트 추출

- tag.text 또는 tag.get_text()

정적 크롤링 : BeautifulSoup 라이브러리를 이용한 웹 데이터 수집

- 수집 목표 : '물고기는 존재하지 않는다'의 쪽수 정보 수집하기
- 수집 과정

1) 해당 정보가 존재하는 웹 페이지 HTML 가져오기

① Yes24 웹사이트에서 해당 도서의 쪽수 정보가 존재하는 웹 페이지로 이동

- Yes24 웹사이트에서 해당 도서 검색
- Yes24 웹사이트에서 해당 도서 상세 페이지 링크 클릭

② Yes24 웹사이트에서 해당 도서의 쪽수 정보가 존재하는 웹 페이지의 url 확인 → 호출 url 작성

③ requests.get(url) 함수 이용 → 도서 검색 결과에 대한 웹 페이지 HTML 획득

정적 크롤링 : BeautifulSoup 라이브러리를 이용한 웹 데이터 수집

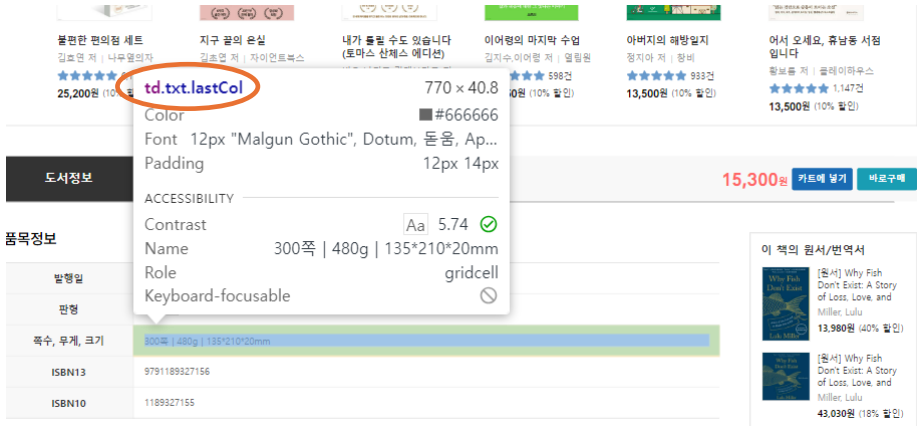
- 수집 과정

2) BeautifulSoup 라이브러리 이용하여 쪽수 정보 가져오기

① BeautifulSoup : HTML 문서를 탐색해서 원하는 부분만 쉽게 뽑아낼 수 있는 기능을 제공하는 라이브러리

② 사용법

```
# 클래스 함수 임폴트 : from bs4 import BeautifulSoup
# BeautifulSoup 객체 생성 : soup = BeautifulSoup(html, 분석도구)
# 쪽수 정보가 들어있는 태그 정보 가져오기 : select() 사용
  tags = soup.select('td.txt.lastCol')
```



정적 크롤링 : BeautifulSoup 라이브러리를 이용한 웹 데이터 수집

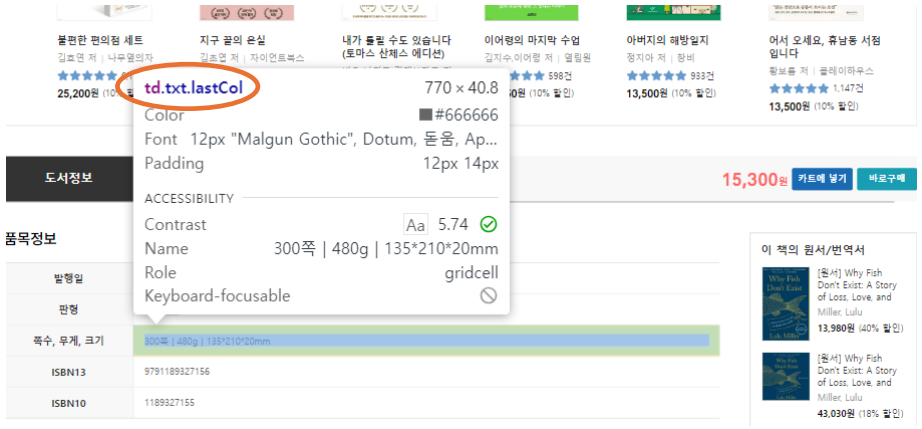
- 수집 과정

2) BeautifulSoup 라이브러리 이용하여 쪽수 정보 가져오기

① BeautifulSoup : HTML 문서를 탐색해서 원하는 부분만 쉽게 뽑아낼 수 있는 기능을 제공하는 라이브러리

② 사용법

```
# 클래스 함수 임폴트 : from bs4 import BeautifulSoup
# BeautifulSoup 객체 생성 : soup = BeautifulSoup(html, 분석도구)
# 쪽수 정보가 들어있는 태그 정보 가져오기 : select() 사용
  tags = soup.select('td.txt.lastCol')
```

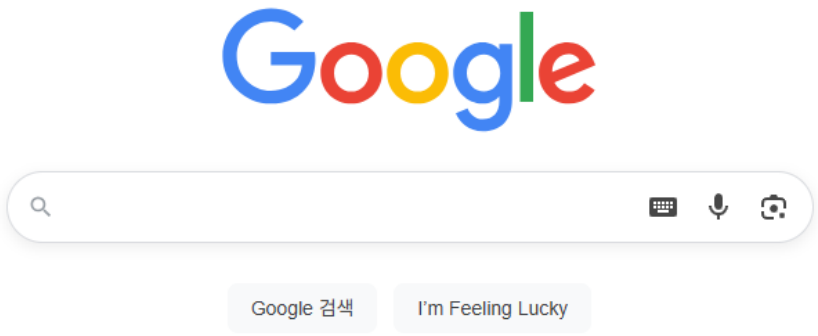


정적 크롤링 : BeautifulSoup 라이브러리를 이용한 웹 데이터 수집 실습

동적 크롤링 : selenium 라이브러리를 이용한 웹 데이터 수집

- 개발자 도구 (F12)
  1) 크롬 브라우저가 웹 페이지의 내부 구조(HTML 코드)를 보여주는 기능
  2) F12 키를 누르면 바로 열림

- 우리가 사용할 단 하나의 기능: ''요소 선택 아이콘'
  1) 개발자 도구 좌측 상단의 '화살표 아이콘'을 클릭
  2) 웹 페이지에서 내가 가져오고 싶은 정보에 마우스를 올리고 클릭
  3) 개발자 도구 창에 해당 정보의 HTML 코드가 빨간 색으로 강조 표시됨





동적 크롤링 : selenium 라이브러리를 이용한 웹 데이터 수집

- '정보의 주소' 찾는 법: Copy selector

  1) 복잡한 HTML 코드를 직접 분석할 필요가 없음

  2) 가장 쉬운 방법

  ① F12 → "요소 선택 아이콘"으로 원하는 정보의 HTML 요소 찾기

  ② 강조 표시된 코드 위에서 마우스 오른쪽 버튼 클릭

  ③ Copy → Copy selector 메뉴 클릭(가장 많이 사용)

  ④ 메모장에 붙여넣기: #input → 해당 요소에 접근할 수 있는 '주소'가 복사됨.

  3) 'selector' 주소: Selenium에게 "이 주소로 가서 정보를 가져와!"라고 명령할 때 사용하는 핵심 키(key)가 됨

동적 크롤링 : selenium 라이브러리를 이용한 웹 데이터 수집

- 실전! Selenium 핵심 코드 3단계

1) 1단계: 필요한 패키지 /도구 임포트 및 '아바타 브라우저' 생성

```
# 필요한 패키지 / 도구 임폴트
from selenium import webdriver
from selenium.webdriver.common.by import By

# Chrome 브라우저를 자동 조작하기 위한 '아바타 브라우저' 생성
driver = webdriver.Chrome()
```

동적 크롤링 : selenium 라이브러리를 이용한 웹 데이터 수집

- 실전! Selenium 핵심 코드 3단계

  2) 원하는 페이지로 이동하기

```
# 이동하고 싶은 웹사이트 주소
url = 'https://product.kyobobook.co.kr/detail/S000001925800'

# 페이지 이동
driver.get(url)
```

동적 크롤링 : selenium 라이브러리를 이용한 웹 데이터 수집

- 실전! Selenium 핵심 코드 3단계

  3) 정보 찾고 가져오기 (Copy selector 활용)

```
#  F12로 복사한 selector 주소
# selector = '#bookBasicInfo > section > div.border-t-1 > table > tbody > tr:nth-child(3) > td > div > span'
selector = '#bookBasicInfo span'

# selector 주소에 해당하는 '요소(element)'를 찾아서 변수에 저장
element = driver.find_element(By.CSS_SELECTOR, selector)

# 찾은 요소에서 text 추출
page_info = element.text.split(' |')[0]

print(f"도서 쪽수: {page_info}")
```

동적 크롤링 : selenium 라이브러리를 이용한 웹 데이터 수집

- 기본 개념 정리: webdriver 패키지

1) 웹 브라우저를 '자동 조작'하고 데이터를 수집하기 위한 핵심 패키지

```
# Chrome 브라우저를 자동 조작하기 위한 '아바타 브라우저' 생성
driver = webdriver.Chrome()
```

2) 웹 브라우저를 실행하고, 특정 URL로 이동하는 기능 제공 → driver.get(url) 함수

3) 웹 페이지 내의 다양한 요소(검색 창, 링크 등)를 찾고, 이러한 요소들과 상호작용할 수 있는 기능 제공 → driver.find_element(By.CSS_SELECTOR, selector)

4) 웹 페이지 내의 특정 요소로부터 정보를 추출하는 기능 제공

→ driver.find_element(By.CSS_SELECTOR, selector).text

동적 크롤링 : selenium 라이브러리를 이용한 웹 데이터 수집

- 수집 목표 : '물고기는 존재하지 않는다'의 쪽수 정보 수집하기

생성형 AI를 이용한 동적 크롤링 실습

- 수집 목표: 마켓컬리 "[사미헌] 갈비탕" 사용자 리뷰 1 page ~ 20 page 에서 상품명, 리뷰, 날짜 정보 수집 및 CSV 파일로 저장하기

- 수집 과정
  1) 생성형 AI에 전달할 정보 수집: 예) 수집 항목별 HTML 요소 이미지
  2) 생성형 AI를 이용하여 수집 및 CSV 파일 생성 프롬프트 작성
  3) 생성된 코드 실행 및 검증

생성형 AI를 이용한 동적 크롤링 실습


MISSION:
COMPLETE